

Trinity: A Scalable and Forward-Secure DSSE for Spatio-Temporal Range Query

Zhijun Li¹, Kuizhi Liu¹, Minghui Xu¹, *Member, IEEE*, Xiangyu Wang, *Member, IEEE*,
Yinbin Miao¹, *Senior Member, IEEE*, Jianfeng Ma¹, *Member, IEEE*, and Xiuzhen Cheng¹, *Fellow, IEEE*

Abstract—Cloud-based outsourced Location-based services significantly impact various aspects of daily life but also raise security concerns. Existing secure retrieval schemes for spatio-temporal data exhibit significant shortcomings regarding dynamic updates; they either compromise privacy through information leakage during updates (lacking forward security) or incur excessively high update costs, hindering practical application. To address these limitations, we first propose a basic filter-based spatio-temporal range query scheme Trinity-I that supports low-cost dynamic updates and automatic expansion. Furthermore, to improve security, reduce storage cost, and false positives, we propose a forward secure and verifiable scheme Trinity-II that simultaneously minimizes storage overhead. Formal security analysis demonstrates that both Trinity-I and Trinity-II achieve Indistinguishability under Selective Chosen-Plaintext Attack (IND-SCPA). Finally, extensive experiments demonstrate that our design Trinity-II significantly reduces storage requirements by 80%, enables data retrieval at the 1 million-record level in just 0.01 seconds, and achieves 10× update efficiency than state-of-art.

Index Terms—Location-based services, spatio-temporal data, dynamic update, forward-secure.

I. INTRODUCTION

THE popularity of smart devices has accelerated Location-Based Services (LBS), which retrieve and sort information based on user location and requests, then provide recommendations to users. In traditional spatial LBS, users can ask if a stationary object sits within a specific area. However, how do users track the movement patterns of dynamic objects

such as vehicles? That is where spatio-temporal LBS comes in, stepping up from mere spatial snapshots to track and query objects across both space and time.

For spatio-temporal LBS service providers, outsourcing spatio-temporal data to the cloud is a practical method way of improving the service quality and reducing local operating costs. However, outsourcing data to third parties inevitably brings security and privacy issues. For example, in 2018, fitness tracking app Strava published a global heat map that visualized the activity of its users, inadvertently revealing the locations and activity time of military personnel at sensitive facilities around the world [1]. Dynamic Searchable Symmetric Encryption (DSSE) [2] offers a potential solution, allowing clients holding specific keys to query and update ciphertext without revealing sensitive data. However, most DSSE schemes permit certain information leakage in exchange for efficiency [3], [4], [5], [6], [7], [8], [9], [10]. Consequently, attackers might infer the content of past queries by strategically inserting new documents and observing matches identified by the server against previous search queries [11] (known as file injection attacks). To mitigate this risk, several studies [12], [13], [14], [15] have introduced forward security, aiming to prevent leakage resulting from updates.

When applying DSSE to spatio-temporal data, scalability becomes a critical issue. The continuous growth of spatio-temporal datasets poses significant challenges for system design. The scalability of DSSE schemes often depends heavily on their underlying index structures, which can be broadly categorized as tree-based [12], [16] and linear index structures [17], [18]. Tree-based spatio-temporal range query schemes inherently face scalability challenges [19], [20]; as data volume increases, frequent tree expansions involving height adjustments and node reorganization become necessary, hindering scalability [21].

In contrast, filter-based schemes, often relying on array-based structures [9], generally offer better scalability. Filters also demonstrate superior space efficiency compared to structures like R-trees [22]. These advantages have fueled the adoption of filters in spatio-temporal range query schemes. Filter-based DSSE schemes for spatio-temporal range queries typically work by transforming high-dimensional spatio-temporal data into one-dimensional representations stored within the filter structure. Leveraging hash functions that map elements to multiple locations within the filter, queries can often be executed via simple, low-complexity bit-wise operations [22]. This approach exploits the fast lookup capabilities

Received 3 January 2025; revised 10 April 2025; accepted 25 April 2025. Date of publication 2 May 2025; date of current version 12 May 2025. This work was supported in part by the National Key Research and Development Program of China under Grant 2022YFB4501000; in part by the National Natural Science Foundation of China under Grant 62232010, Grant 62302266, Grant 62202364, Grant U23A20302, and Grant U24A20244; in part by Shandong Science Fund for Excellent Young Scholars under Grant 2023HWYQ-008; in part by Shandong Science Fund for Key Fundamental Research Project under Grant ZR2022ZD02; in part by the Fellowship of China National Postdoctoral Program for Innovation Talents under Grant BX20230279; in part by China Postdoctoral Science Foundation under Grant 2024M752534; and in part by the Key Research and Development Program of Shaanxi under Grant 2024GX-YBXM-075. The associate editor coordinating the review of this article and approving it for publication was Dr. Yang Zhang. (Corresponding author: Minghui Xu.)

Zhijun Li, Minghui Xu, and Xiuzhen Cheng are with the School of Computer Science and Technology, Shandong University, Qingdao 266237, China (e-mail: richikun2014@gmail.com; mhxu@sdu.edu.cn; xzcheng@sdu.edu.cn).

Kuizhi Liu, Xiangyu Wang, Yinbin Miao, and Jianfeng Ma are with the School of Cyber Engineering, Xidian University, Xi'an 710071, China (e-mail: ighxy@163.com; wangxiangyu01@xidian.edu.cn; ybmiao@xidian.edu.cn; jfma@mail.xidian.edu.cn).

Digital Object Identifier 10.1109/TIFS.2025.3566552

of filters to efficiently determine element existence and process spatio-temporal range queries. While many DSSE schemes have been developed based on Bloom filters [9], [14], [22], [23], [24], several challenges remain.

A. Challenges

Challenge I: How to construct a filter that enables flexible and efficient element deletion. Existing range query schemes based on Bloom filters [5], [22], [23], [24] generally do not support efficient deletion. This limitation arises because standard Bloom filters use a bit array where multiple elements may map to and share the same bit positions [25]. Clearing a bit to delete one element can inadvertently affect the representation of other elements sharing that bit [26]. For example, deleting “seafood” might accidentally remove “Japanese cuisine” if they share bit positions. While some non-filter DSSE schemes [12], [14] support deletion, our experimental results indicate their efficiency can be a concern, and they are not directly applicable to filter-based architectures.

Challenge II: How can the scalability of forward-secure filters be improved. Existing forward-secure range query filters [5], [14], [23] face scalability problems. As the volume of spatio-temporal data grows, the probability of hash collisions within the filter increases, negatively impacting query efficiency and the false positive rate [24]. Furthermore, if the filter reaches its maximum capacity, adding new entries may necessitate rebuilding the entire structure. Filter parameters (such as capacity) are often static once initialized [27], and expansion typically requires costly remapping of all existing data [25]. While initial over-provisioning might seem like a solution, it leads to significant wasted space, especially if data insertion rates are variable, as periods of inactivity exacerbate this inefficiency.

Challenge III: How to overcome the trade-off between saving storage and minimizing false positives. Filter-based schemes [28], [29], [30] need free space to keep a low false positive rate (FPR). For example, the FPR of widely used Bloom filters is given by $\epsilon = (1 - e^{-k \cdot \frac{n}{m}})^k$, where k is the number of hash functions, m is the length of the filter, and n is the number of inserted elements [31]. To achieve a practical FPR of 0.01%, a balance between accuracy and efficiency is sought [25]. This configuration inherently sets the ratio $\frac{m}{n} = \frac{k}{\ln 2}$, where $k = -\frac{\ln p}{\ln 2}$. This fixed ratio imposes a fundamental design constraint, requiring $\frac{m}{n} \geq 20$ to maintain the desired FPR. In other words, at least 19 times the number of inserted elements in free space is necessary. Overcoming this trade-off is crucial for improving filter-based DSSE.

Our Contributions. We introduce two novel DSSE schemes tailored for spatio-temporal range queries.

- **Trinity-I.** A fundamental DSSE construction supporting dynamic and efficient element updates (addition and deletion). Trinity-I offers a performance boost over state-of-the-art methods, achieving up to a 20× speedup. And Trinity-I adaptively expands its index structure, so we can keep a negligible false positive rate and low latency on search and update for the filter. Besides, our fundamental DSSE construction is secure against IND-SCPA.

- **Trinity-II.** Building upon Trinity-I, this scheme provides additional high accuracy, and forward security. Trinity-II offers a verification function that strikes a balance between storage efficiency and low false positives. It reduces storage requirements by 80% while enabling data retrieval at the 1 million-record level in just 0.01 seconds, and offers enhanced forward security against file injection attacks. Trinity-II still offers a performance boost over state-of-the-art methods, achieving up to a 10× speedup on update latency.

II. RELATED WORK

In this section, we introduce some related work. To ensure a more impartial and lucid presentation, we list Trinity with previous works in Table I.

DSSE: DSSE is a cryptographic technique that enables clients to securely outsource encrypted databases to servers while still maintaining the ability to perform search and update operations on encrypted data [2], [14], [16], [32], [33], [34], [35]. While DSSE provides efficient access to encrypted databases, it inevitably incurs some information leakage, particularly during update operations, where sensitive data can potentially be exposed, thus compromising privacy. The concept of forward security, introduced by Stefanov *et al.* [2], aims to address this issue by ensuring that updates to the encrypted database do not reveal information about previous queries or data. Zhang *et al.* [11] further highlighted the vulnerability of DSSE to leakage attack, specifically file injection tactics, where an attacker strategically inserts a limited number of documents into the encrypted database to deduce search queries. Following this finding, Bost [36] provided a formal definition of forward security and proposed a concrete scheme that incorporates forward security in DSSE. Since then, several forward-secure DSSE schemes have been proposed [16], [37], [38], [39], with a focus on improving security, efficiency, and functional diversity.

Filter-Based DSSE: The common drawback of filter-based search DSSE schemes [5], [9], [23], [24], [28], [29], [30], particularly Bloom filter-based schemes, is that they cannot support deletion. These filters are designed to efficiently test membership in a set while allowing for false positive results, meaning they can indicate that an element is present in the set when it is not. This characteristic is inherent in their probabilistic nature, which prioritizes space efficiency over absolute accuracy. Wang *et al.* [40] introduced a pioneering privacy-preserving circular range search scheme; however, the search token size scales with the square of the search radius R , posing scalability challenges. To address these issues, Wang *et al.* [23] ingeniously leveraged the Hilbert curve, effectively reducing queries of the two-dimensional spatial range to one-dimensional searches, thus significantly enhancing retrieval speed. Although this approach represents a notable improvement, it still lacks support for dynamic updates and is limited to a fixed data scale. Similarly, the non-scalable structure employed in [9] and [24] requires significant space overhead to maintain a low FPR, further highlighting the limitations of these methods.

None-Filter DSSE: Balancing efficiency and security has consistently presented a challenge in privacy-preserving LBS [6], [7], [41], [42]. Various effective privacy-preserving

TABLE I
COMPARISON WITH PRIOR WORKS

Schemes	Cryptographic Primitive	Dynamic Update	Forward Security	High Efficiency	Spatio-Temporal	Scalability	Security Model
DSSE _{SKQ} [14]	ASHE	✓	✓	✓	×	×	IND-CPA
SKSE-II [23]	HVE	×	×	✓	×	×	IND-SCPA
GRS-II [12]	ASHE	✓	✓	×	×	×	IND-CPA
Trinity-I	SHVE	✓	×	✓	✓	×	IND-SCPA
Trinity-II	SHVE	✓	✓	✓	✓	✓	IND-SCPA

Notes. ASHE stands for Additive Symmetric Homomorphic Encryption. ASPE stands for Asymmetric Scalar Product-Preservation Encryption. HVE stands for Hidden Vector Encryption. SHVE stands for Symmetric-key Hidden Vector Encryption.

approaches have been proposed, including the use of Order-Revealing Encryption (ORE) [41], and a variant of Order-Preserving Encryption (OPE) [6]. However, recent research [42] has highlighted severe security vulnerabilities in property-preserving encryption schemes such as OPE and ORE. Cui *et al.* [22] contributed a privacy-preserving Boolean Range Query (BRQ) solution, which was later found to be vulnerable to Ciphertext-Only Attacks (COA) [43] due to the use of Asymmetric Scalar Product-Preservation Encryption (ASPE) [44]. To address this security issue, Yang *et al.* [8] proposed an enhanced ASPE scheme that achieves Indistinguishability under Chosen Plaintext Attack (IND-CPA). However, their approach compromised on search efficiency. Kermanshahi *et al.* [12] proposed a forward-secure DSSE for spatial range query, but the efficiency of their solution is unacceptable. Wang *et al.* [23] then improved search speed by incorporating a Bloom filter hierarchical tree, but their solution was limited to single-user scenarios due to its symmetric key setting and focused solely on spatial queries, neglecting the temporal dimension. Li *et al.* [15] developed an efficient privacy-preserving spatio-temporal LBS scheme capable of retrieving million-level data in milliseconds.

III. PRELIMINARIES

Here, we introduce some crypto primitives and data structures widely used in Trinity including Hilbert curve, Bloom filter, and SHVE.

A. Hilbert Curve

A Hilbert curve is a type of continuous fractal space-filling curve that fills a d -dimensional area [45]. For a d -dimensional space, each dimension is uniformly divided into 2^h segments, where h denotes the order of the Hilbert curve. The Hilbert curve is constructed by recursively dividing an area into 2^d smaller areas and connecting the centers of these smaller areas in a specific order. With the Hilbert curve, any d -dimensional spatial range query can be converted into a range query in one-dimensional space of 2^d consecutive areas, each represented by a dh -bit value.

B. Quotient Filter

A quotient filter is a variant of the Bloom filter. It can quickly detect whether an element is in a set, indicating that the data definitely do not exist or may possibly exist. The quotient filter employs a space-efficient hash table mechanism,

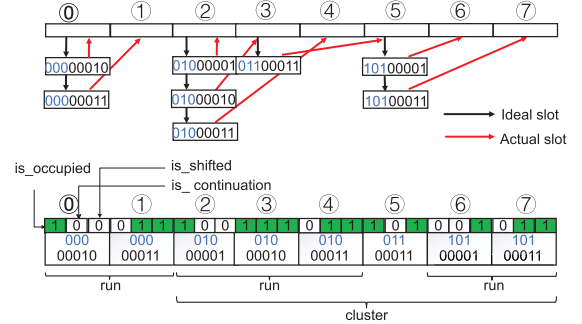


Fig. 1. An example of quotient filter.

dividing the p -bit fingerprint of an element into two distinct segments. The first segment, known as the quotient, consists of the q most significant bits and is used to rapidly determine the “target position” of the element within the filter. The second segment, referred to as the remainder, encapsulates the $r = p - q$ least significant bits and serves to differentiate elements with the same quotient. The original slot where the hash output point’s quotient resides is called the canonical slot. A sequence of consecutive slots containing remainders with the same quotient is termed a run. A cluster in a quotient filter is a contiguous sequence of runs, commencing with the first run whose initial fingerprint occupies its canonical slot, and extending until an unoccupied slot is encountered or the another run occupies its canonical slot is identified. Each slot contains a 3-bit counter.

- **Is_occupied.** Set to 1 if a slot is occupied correctly, that is, a quotient value corresponds to the slot index.
- **Is_continuation.** Set to 1 when a slot is not the start of the run.
- **Is_shifted.** Set to 1 when this slot is not the start of the cluster. It happens when there is an offset between the position where the remainder is stored and the index represented by the quotient associated with the remainder.

We provide an example of the quotient filter in Fig. 1. The upper portion depicts a simplified view, where eight fingerprints are categorized into buckets based on their quotient (blue numbers). For example, the first fingerprint 000|00010, with the quotient of 000, is placed in slot ①. The second fingerprint 000|00011 ideally belongs to slot ①, but slot ① is occupied, so we shift it right to slot ② (the actual slot of the second fingerprint). The lower portion shows the actual storage mechanism. Each bucket holds a 3-bit counter and a remainder. In this example, 000|00010, and 000|00011 share

the same quotient, forming a **run**. The eight fingerprints finally constitute three **runs** and one **cluster**.

C. Symmetric-Key Hidden Vector Encryption

Symmetric-key Hidden Vector Encryption (SHVE) is a lightweight crypto primitive based on Hidden Vector Encryption (HVE). Like HVE, it supports conjunctive, equality, and subset queries on encrypted data. Let Γ be a finite field and $*$ be a wildcard symbol not belong to Γ , $\Gamma_* = \Gamma \cup \{*\}$. The SHVE is defined in the subsequent algorithm:

- **SHVE.Setup**($\{0, 1\}^\lambda$): On input a security parameter λ , it returns a primary secret key msk and the message space $\mathcal{M}$.
- **SHVE.KeyGen**($msk, \mathbf{V} \in \Gamma_*^d$): On input a primary secret key msk and a predicate vector $\mathbf{v} = \{v_1, \dots, v_d\} \in \Gamma_*^d$, it returns the s as a decryption key.
- **SHVE.Enc**($msk, \mu \in \mathcal{M}, \mathbf{ind} \in \Gamma_*^d$): On input an index vector $\mathbf{ind}$, a primary key msk and a message μ , it returns the ciphertext c associated with $(\mathbf{ind}, \mu)$.
- **SHVE.Query**(c, s): On input a ciphertext c associated with $(\mathbf{ind}, \mu)$ and a decryption key s corresponding to the predicate vector $\mathbf{v}$, it returns μ if $\mathcal{P}_\mathbf{v}^{\text{SHVE}}(\mathbf{x}) = 1$; else returns null.

IV. PROBLEM FORMULATION

In this section, we illustrate the system model and the threat model considered in this paper.

A. System Model

Trinity involves three entities: Cloud Server (CS), Data Owner (DO) and Data Users (DU).

- **Cloud server.** The CS is assumed to have sufficient computing and storage capabilities. The primary responsibilities of the CS include storing the ciphertext and secure index provided by the DO. Upon receiving a search token from a DU, the CS processes the token and returns the results to the DU.
- **Data owner.** The DO encrypts the files and generates the corresponding security index, which is uploaded to the CS. Additionally, the DO generates a secret key and securely distributes it to the authorized DU through a secure channel.
- **Data users.** The DU obtains the key from the DO and creates search tokens corresponding to the spatio-temporal objects being queried. These tokens are then sent to the CS. DU receives the results from the CS after the search is completed.

A spatio-temporal database **DB** is defined as: $\mathbf{DB} = \{\mathbf{p}_i, \mathbf{ind}_i\}_{i=1}^N$, $\mathbf{p}_i = (x_i, y_i, z_i)$ is a space-time node, $\mathbf{ind}_i$ is a file identifier, and N is the size of the quotient filter.

Definition 1: For an encrypted spatio-temporal database **EDB** and a search token ST , Trinity is to search a subset $\{\mathbf{ind}_i^*\}_{i=1}^c$ such that $\forall 1 \leq i \leq c, \mathcal{P}_i^* \in \mathcal{R}_Q$, where $\mathcal{P}_i^*$ represents the minimum set of prefix elements that collectively cover the entire range query $\mathcal{R}_Q$.

Definition 2: The Trinity is a DSSE scheme consisting three algorithms described as follows.

- **Setup**($\{0, 1\}^\lambda$) $\rightarrow (K_\Sigma, \sigma; \text{EDB})$: On input a security parameter λ , it returns a secret key set K_Σ , a state σ and the encrypted database **EDB**.
- **Search**($Q, K_\Sigma, \sigma; \text{EDB}$) $\rightarrow R$: On input a query Q , a secret key set K_Σ , a state σ and the encrypted database **EDB**, the client sends a search request to the server, and the server returns the result R after searching all over the **EDB**.
- **Update**($K_\Sigma, O, up, \sigma; \text{EDB}$) $\rightarrow (K_\Sigma, \sigma'; \text{EDB}')$: On input ($K_\Sigma, O, up, \sigma; \text{EDB}$), where $O = \{p, \mathbf{ind}\}$ and $up \in \{\text{add}, \text{del}\}$. up , add and del denote update, addition, and deletion, respectively. The server adds or deletes the object O from **EDB**.

B. Threat Model

We assume that the Data Owner (DO) and Data User (DU) are trustworthy. The Cloud Server (CS), however, is considered honest-but-curious, meaning it faithfully executes the protocol but may attempt to extract information from encrypted data. To ensure the security of the DSSE system (e.g., Trinity), the adversary, represented as $\mathcal{A}$, should not be able to gain any meaningful information from the encrypted databases or queries. This security property is formalized through a real-world vs. ideal-world game-based approach. Let $\mathcal{L} = \{\mathcal{L}^{\text{Stp}}, \mathcal{L}^{\text{rch}}, \mathcal{L}^{\text{Updt}}\}$ denote the leakage function that captures the information an adversary can potentially obtain from the **Setup**, **Search**, and **Update** phases. The formal definitions of these games are as follows:

- The real game $\text{Game}_\mathcal{A}^{\mathcal{R}}(\lambda)$: $\mathcal{A}$ selects a database **DB** and generates the **EDB** via **Setup**. Subsequently, $\mathcal{A}$ adaptively runs **Search** or **Update**. Throughout the experiment, $\mathcal{A}$ maintains full observational access to the real operational transcripts. Upon conclusion of the adaptive query phase, $\mathcal{A}$ outputs $b \in \{0, 1\}$.
- The ideal game $\text{Game}_\mathcal{A}^{\mathcal{S}}(\lambda)$: $\mathcal{A}$ selects a database **DB** and generates the **EDB** via simulator $\mathcal{S}(\mathcal{L}^{\text{Stp}}(\mathbf{DB}))$. Subsequently, $\mathcal{A}$ adaptively runs $\mathcal{S}(\mathcal{L}^{\text{rch}})$ or $\mathcal{S}(\mathcal{L}^{\text{Updt}})$. Throughout the experiment, $\mathcal{A}$ maintains full observational access to the simulated operational transcripts. Upon conclusion of the adaptive query phase, $\mathcal{A}$ outputs $b \in \{0, 1\}$.

If there is an efficient simulator $\mathcal{S}$ such that:

If $\mathcal{A}$ cannot distinguish between the real-world game $\text{Game}_\mathcal{A}^{\mathcal{R}}(\lambda)$ and the ideal-world game $\text{Game}_\mathcal{A}^{\mathcal{S}}(\lambda)$, then the system is considered secure, meaning no information beyond the specified leakage function is leaked.

Definition 3: A DSSE is $\mathcal{L}$ -adaptively secure [46] if for any probabilistic polynomial-time (PPT) adversary $\mathcal{A}$, there is an efficient simulator $\mathcal{S}$:

$$|\Pr[\text{Game}_\mathcal{A}^{\mathcal{R}}(\lambda)=1] - \Pr[\text{Game}_\mathcal{A}^{\mathcal{S}}(\lambda) = 1]| \leq \text{negl}(\lambda).$$

Besides, forward security plays a pivotal role in safeguarding DSSE schemes from leakage-abuse attacks. The DSSE scheme is considered ‘forward-secure’ if there is no connection between an update of encrypted data and any previously performed search results. Formally speaking,

Definition 4: A $\mathcal{L}$ -adaptively secure DSSE is forward-secure [36] if the update leakage function $\mathcal{L}^{Updt}$ is defined as follows,

$$\mathcal{L}^{Updt}(op, in = (p, ind)) = \mathcal{L}'(up, in = (ind, c)),$$

where op denotes the operation like addition or deletion, in denotes the input, ind represents the document identifier, and c is the number of update files.

C. Design Goals

The design goals of Trinity are described as follows.

- **Dynamic.** The proposed scheme is designed to be dynamically configurable, enabling document addition and deletion operations.
- **Update-efficient.** The proposed scheme aims to achieve more efficient updates compared to existing forward-secure schemes.
- **Scalable.** The proposed scheme aims to be more scalable, which means it can efficiently expand its capacity for a continuous stream of data.
- **Verifiable.** The proposed scheme aims to be verifiable, which means there would be no false positives in results and no storage waste to keep false positives minimized.
- **Privacy-preserving.** The proposed scheme aims to be $\mathcal{L}$ -adaptively secure and forward-secure. The system should safeguard sensitive information such as file collections, indexes, and background information of keywords from unauthorized access. Furthermore, our scheme adheres to forward security principles, ensuring that the CS remains oblivious to any association between recent updates and prior search results.

V. TRINITY SCHEMES

In this section, we first introduce a scalable, update-efficient spatio-temporal range search scheme, Trinity-I, that builds on quotient filter, Hilbert curve and SHVE. Subsequently, we propose a forward-secure, and verifiable scheme Trinity-II. Trinity-I is faster than Trinity-II in search and update latency, but it wastes more storage and lacks forward security.

A. Trinity-I: Basic Trinity Construction

1) *Technique Overview:* We treat spatio-temporal data as three-dimensional data and use the Hilbert curve to reduce it to a one-dimensional form. This technique effectively translates spatio-temporal range queries in a multi-dimensional space into multiple one-dimensional range queries. We denote the Hilbert curve of a point p or a range R as $\mathcal{H}(p)$ and $\mathcal{H}(R)$, respectively. Although it is feasible to encode all elements within a specified range into a QF and test for the presence of an encoded space-time node, the QF's size scales linearly with the number of elements. Notably, range queries typically encompass significantly more elements than individual data objects. To optimize QF size, we must minimize the number of elements involved in range queries. To address this challenge, we employ a prefix membership verification technique introduced by Liu *et al.* [47].

Previously mentioned Bloom filter-based schemes [9], [24], [28], [29], [30] suffer from several challenges: deletion capability, scalable structure, and minimizing false positives. In response to these limitations, our approach leverages the quotient filter, and scalable Bloom filter technology to resolve those challenges. The quotient filter employs a fingerprint-based storage mechanism, where each element is represented by a unique fingerprint comprising quotient and remainder components, thereby facilitating precise element identification and efficient deletion operations.

The system's performance inversely correlates with data volume: as data volume increases, both insertion and retrieval operations become less efficient, while false positive rates increase. So we adjust the structure of the quotient filter to make it scalable, i.e. when the amount of inserted data approaches a certain threshold (affecting performance or false positive rate is high), it automatically expands dynamically. We simply borrow one bit from the remainder into the quotient instead of rehashing all elements for the expanding.

To ascertain the subset relationship between encrypted vector representations, we leverage SHVE for performing privacy-preserving set membership queries within a cryptographically secured computational domain [48]. Specifically, our scheme uses multi-threaded computing technology to speed up SHVE.

2) *Details of Basic Trinity Construction:* In this section, we will briefly introduce **Setup** and **Search**, focusing on dynamic **Update**. Here dynamic **Update** refers to the ability to efficiently modify the data structure that supports the encryption scheme, allowing for the addition and deletion of elements without requiring a complete re-encryption of the dataset. Specially, our scheme uses the murmur hash function and multi-threaded computing SHVE.

Setup $\{0, 1\}^L \rightarrow (K_\Sigma, \sigma; \text{EDB})$: Given a security parameter λ and a database DB, it returns a primary key msk , t hash functions $H = \{H_i(\cdot)\}_{i=1}^t$, and an encrypted database EDB. And DO share the secret key set $K_\Sigma = \{msk, H\}$ with the DU. For each space-time node $O_i = \{p_i, ind_i\}$, encoding the spatio-temporal data.

For a given ω -bits data item $X = a_1 a_2 \cdots a_\omega$, its prefix family is defined as a set of $\omega + 1$ elements: $P(X) = \{a_1 a_2 \cdots a_\omega, a_1 a_2 \cdots a_{\omega-1} *, \dots, a_1 * \cdots *, * * \cdots *\}$, where the i -th prefix element is $a_1 a_2 \cdots a_{\omega-i+1} * \cdots *$. Given a range $[x_{min}, x_{max}]$, the query $\mathcal{P}([x_{min}, x_{max}])$ represents the minimum set of prefix elements that collectively cover the entire range. An item X belongs to the range $[x_{min}, x_{max}]$ if and only if the intersection of its prefix family $P(X)$ and the query prefix set $\mathcal{P}([x_{min}, x_{max}])$ is non-empty: $X \in [x_{min}, x_{max}] \Leftrightarrow P(X) \cap \mathcal{P}([x_{min}, x_{max}]) \neq \emptyset$.

Fig. 2 demonstrates examples of spatio-temporal range queries over a spatio-temporal database utilizing our spatio-temporal data and query encoding methodology. Consider a spatio-temporal database holding four space-time nodes, each encoded via the Hilbert curve and represented as a prefix family. To retrieve objects within region R_1 , the DU initially encodes the query range as $\{[4, 7]\}$ and subsequently generates the corresponding query prefixes as $\{01 * *\}$. Since $001 * **$ belongs to the prefix family of $P(\mathcal{H}(O_1))$, we conclude that

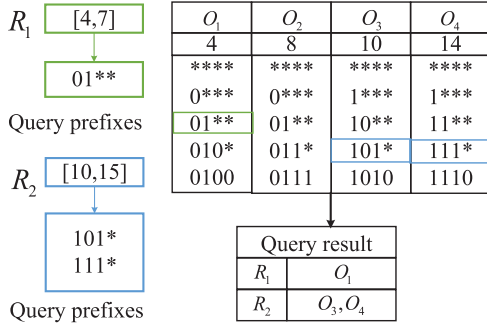


Fig. 2. An example of encoded Hilbert curve with prefixes.

Algorithm 1 Setup (1^A)

Input: $(\{0, 1\}^\lambda, \text{DB})$
Output: $(K_\Sigma, \sigma; \text{EDB})$

- 1 $\text{Setup}(1^\lambda) \rightarrow \text{msk};$
- 2 Randomly generates $H = \{H_i(\cdot)\}_{i=1}^t$;
- 3 $K_\Sigma = \{\text{msk}, H\};$
- 4 **for** space-time node $O_i = \{p_i, \text{ind}_i\}$ **do**
- 5 $H(P(\mathcal{H}(p_i))) \rightarrow e_i;$
- 6 $e_i \rightarrow (F_Q, F_R);$
- 7 $\text{QF}\{H, (F_Q, F_R)\} \rightarrow \text{QF}_i;$
- 8 $\text{SHVE.Enc}(\text{msk}, \text{"True"}, \text{QF}_i) \rightarrow \text{C}_{\text{SHVE}}(\text{QF}_i);$
- 9 $\{\text{C}_{\text{SHVE}}(\text{QF}_i), \text{ind}_i\}_{i=1}^N \rightarrow \text{EDB};$
- 10 **return** $(K_\Sigma, \sigma; \text{EDB});$

O_1 resides within R_1 . To query objects within R_2 , the DU similarly encodes the query range as $\{[10, 15]\}$ and constructs the corresponding query prefixes as $\{101*, 111*\}$. We determine that O_3 and O_4 belong to R_2 based on the presence of $101* \in P(\mathcal{H}(O_3))$ and $111* \in P(\mathcal{H}(O_4))$. We create a quotient filter and store the fingerprints $H(p_i)$ of the space-time node in a trinity form $(H(p_i), F_Q, F_R)$. Finally, we encrypt all the elements within the quotient filter using msk . A formal representation of the **Setup** phase is displayed in Algorithm 1.

Search $(K_\Sigma, Q, \sigma; \text{EDB}) \rightarrow R$: Given a query Q , a secret key set K_Σ , a state σ , and the encrypted database EDB , the DU sends a search request to the server. The CS then returns the result R after searching through the EDB . To search for a value in a Quotient Filter, DU first calculates the fingerprint e_q , and divides it as quotient F_Q and remainder F_R , and encrypts them with msk . Then DU sends the search token $\text{S}_{\text{SHVE}}(\text{QF}_Q)$ to the CS. The CS checks if the target slot corresponding to T_{F_Q} is occupied. If it is occupied, find the start index of the run corresponding to the quotient F_Q . The CS continues the loop when the current slot is a continuation of run. The CS starts from the starting position s , compares the remainder one by one. If the remainder is equal to F_R , return true. If the remainder is greater than F_R , return false. If there is no matched remainder in run, return false. A formal representation of the **Search** phase is displayed in Algorithm 2.

Update $(K_\Sigma, O, \text{up}, \sigma; \text{EDB}) \rightarrow (K_\Sigma, \sigma'; \text{EDB}')$: Given $(K_\Sigma, O, \text{up}, \sigma; \text{EDB})$, the CS adds or deletes the object O from the EDB . For both addition and deletion, DO encodes the updated data and uses a hash function to obtain the updated fingerprint e_a or e_d (for addition or deletion). DO

Algorithm 2 Search

Input: $(K_\Sigma, Q, \sigma; \text{EDB})$
Output: R

- 1 **DU:**
- 2 $H(P(\mathcal{H}(R))) \rightarrow e_q, e_q \rightarrow (F_Q, F_R);$
- 3 $\text{QF}\{H, (F_Q, F_R)\} \rightarrow \text{QF}_Q;$
- 4 $\text{SHVE.KeyGen}(\text{msk}, \text{QF}_Q) \rightarrow \text{S}_{\text{SHVE}}(\text{QF}_Q);$
- 5 Send $\text{S}_{\text{SHVE}}(\text{QF}_Q)$ to the CS;
- 6 **CS:**
- 7 **if** $\text{is_occupied}(T_{F_Q}) == \text{false}$ **then**
- 8 **return** false /* Checks if the element at the quotient index is occupied. If not, it returns false immediately. */
- 9 $s = \text{find_run_index}(\text{QF}, F_Q);$ /* Finds the starting index of the run associated with the quotient. */
- 10 **do**
- 11 $\text{rem} = \text{get_remainder}(\text{get_elem}(\text{QF}, s));$
- 12 **if** $\text{rem} == F_R$ **then**
- 13 **return** true;
- 14 **else**
- 15 $\text{rem} > F_R;$
- 16 **return** false;
- 17 $s = \text{incr}(\text{QF}, s);$ /* Increments the index to access the next element in the run. */
- 18 **while** $(\text{is_continuation}(\text{get_elem}(\text{QF}, s)));$
- 19 **return** false;

then computes the quotient and remainder of the fingerprint. Do encrypt the quotient and remainder and sends them to the server for preparation to be inserted into the quotient filter.

As for addition, let us define prev as the element currently at index s , and curr as the element that will be written to index s . If the slot corresponding to the quotient is empty, we add it directly and end. We set the is_occupied metadata bit and find the starting position of the run corresponding to F_Q . Note that the is_occupied metadata bit in F_Q must be marked. If the slot is not empty, the CS executes the loop: set the previous slot as is_shifted , if the previous slot is set as is_occupied , set the current slot curr as is_occupied , and clear the metadata is_occupied of the previous slot prev . If the is_occupied in T_{F_Q} is not marked, then we return the expected starting position of the run (because the run corresponding to F_Q does not exist). Otherwise, we return the starting position of the run (because the run corresponding to F_Q exists). If the run corresponding to F_Q exists, we must determine the specific insertion position to maintain the order of run after insertion. The result is in the variable s . If s is the starting position of the run, we must set is_continuation at the starting position since, after adding the fingerprint at s , this element becomes part of the continuation of the run; otherwise, we must set is_continuation in the element being added. We determine whether the insertion position is a canonical slot. If not, then we set is_shifted in insert_into and then move elements one by one.

When the number of entries in QF_i exceeds 5% of its maximum size, a new QF_{i+1} is created (usually twice the size), the value of quotient q increases by 1 (due to capacity doubling),

Algorithm 3 Update_Addition

Input: (K_Σ, O, add, σ ; EDB)
Output: (K_Σ, σ' ; EDB')

1 **DO:**
2 $H(P(\mathcal{H}(p_a))) \rightarrow e_a, e_a \rightarrow (F_Q, F_R)$;
3 $QF\{H, (F_Q, F_R)\} \rightarrow QF_A$;
4 $SHVE.KeyGen(msk, QF_A) \rightarrow S_{SHVE}(QF_A)$;
5 Send $S_{SHVE}(QF_A)$ to the CS;
6 **CS:**
7 $prev = get_elem(QF, s)$;
8 $empty = is_empty_element(prev)$;
9 **do**
10 **if** (!empty) **then**
11 $prev = set_shifted(prev)$;
12 **if** (is_occupied_element(prev)) **then**
13 $curr = set_occupied_element(curr)$;
14 $prev = clr_occupied_element(prev)$;
15 $set_element(QF, s, curr)$;
16 $curr = prev, s = incr(QF, s), prev = get_elem(QF, s)$
17 ;
17 $empty = is_empty_element(prev)$;
18 **while** (!empty);
19 **if** ($QF_i_entries \geq \frac{1}{20} QF_i_max_size$) **then**
20 $i++$; /* Expand the original QF_i to QF_{i+1} when
20 QF_i exceeds 5% of the capacity. */
21 $T_{F_Q} = get_elem(QF, F_Q)$;
22 $entry = (F_R \ll 3) \& \sim 7$;
23 **if** (is_empty_element(T_{F_Q})) **then**
24 $set_elem(QF, F_Q, set_occupied(entry))$;
25 $QF_entries++$;
26 **return true**
27 **if** (is_occupied(T_{F_Q})) **then**
28 $set_elem(QF, F_Q, set_occupied(T_{F_Q}))$;
29 $start = find_run_index(QF, F_Q)$;
30 $s = start$;
31 **if** $s \neq F_Q$ **then**
32 $entry = set_shifted(entry)$;
33 $insert_into(QF, s, entry)$;
34 $QF_entries++$;
35 **return true**

and the value of remainder r remains unchanged. The total number of bits in the new filter is $p = q_{new} + r$. For example, the quotient and remainder of fingerprint 110|0101 are 110 and 0101. In this old filter, $p = 7, r = 4, q = p - r = 3$. As for the new filter that doubles the size, the fingerprint and remainder stay unchanged. The quotient $q_{new} = q + 1 = 3 + 1 = 4$. So the new quotient is taken as the highest four digits 1100, and the remainder is still 0101. A formal representation of the **Addition** phase is displayed in Algorithm 3.

As for deletion, sp represents the index of the entry after s , $curr$ represents the value of the slot corresponding to s , $next$ represents the value of the slot corresponding to sp , and $next$ represents the original value of s . The CS retrieves the current element from position s in the QF and obtains the next

increment position of s , saving the original starting position. Then it enters an infinite loop to process entry sliding. First, it checks if the current slot $curr$ is occupied. If the next slot is empty or marks the start of a cluster, the current slot $curr$ is set as empty. Otherwise, the CS prepares to update the next slot $next$. If the next slot $next$ marks the start of a run, the CS finds the next occupied quotient F_Q and increments the quotient's position. If the current slot $curr$ is occupied and the quotient equals the current position, the $is_shifted$ flag is cleared.

The CS keeps the slot $is_occupied$ and updates the position pointer sp , incrementing both the pointer sp and the current slot $curr$. If the start of a run for the quotient is found at position s , the CS traverses consecutive slots to retrieve the remainder of the current slot $curr$. If a matching remainder is rem found, the traversal terminates. If the remainder rem is greater than the target remainder F_R , the CS returns true, indicating that no match was found. The CS then moves s to the next slot and continues until the target remainder is found. Once the target remainder is found, the CS deletes the entry. A formal representation of the **Deletion** phase is displayed in Algorithm 4.

B. Trinity-II: Trinity With Improved Security and Accuracy

1) *Technique Overview:* The basic Trinity we proposed earlier is efficient enough, but as we said before, we want a secure and accurate and dynamic Trinity. So first, we use "salts" and CPRF techniques to avoid leakage from addition. To achieve this, Trinity employs a technique where hashed order tokens, denoted as OT_i , are used to introduce unique "salts" to the encrypted data points. This salting process is streamlined by utilizing a CPRF, which effectively minimizes bandwidth usage. Upon a new node being added to the EDB (specifically, the $(i + 1)$ th addition), a special ordering token, OT_i , is generated by the DO using the secret key K . This token is then used to "salt" the space-time node, enhancing its security. When a search query is initiated, the DUs provide both the ST and the query itself to the CS. The CS, in turn, leverages the ST to compute all necessary ordering tokens, subsequently reversing the salting process to unveil the original encrypted points.

We utilize the quotient filter data structure, a variant of the Bloom filter. Like the Bloom filter, the quotient filter also exhibits a false positive rate. And the FPR is an inherent challenge in filters, stemming from the design philosophy of the filter. It consists of a fixed-size binary bit array and a series of random mapping functions (hash functions). The core idea is to use multiple different hash functions to address conflicts. Due to the issue of hash collisions (where two different elements may map to the same value after applying a hash function), it introduces multiple hash functions to reduce collisions. If any hash function determines that an element is not in the set, then the element is definitely absent. The element likely exists only when all hash functions unanimously indicate its presence.

$$FPR = \left(1 - e^{-k \cdot \frac{n}{m}}\right)^k,$$

Algorithm 4 Update_Deletion

Input: $(K_{\Sigma}, O, del, \sigma; \text{EDB})$
Output: $(K_{\Sigma}, \sigma'; \text{EDB}')$

```

1 DO;
2  $H(P(\mathcal{H}(p_d))) \rightarrow e_d, e_d \rightarrow (F_Q, F_R);$ 
3  $\text{QF}\{H, (F_Q, F_R)\} \rightarrow \text{QF}_D;$ 
4  $\text{SHVE.KeyGen}(msk, \text{QF}_D) \rightarrow \text{SSHVE}(\text{QF}_D);$ 
5 Send  $\text{SSHVE}(\text{QF}_D)$  to the CS;
6 CS:
7  $\text{get\_elem}(\text{QF}, s) \rightarrow \text{curr}, \text{incr}(\text{QF}, s) \rightarrow sp, s \rightarrow \text{orig};$ 
8 while (true) do
9    $\text{is\_occupied}(\text{curr}) \rightarrow \text{curr\_occupied};$ 
10  if ( $\text{is\_empty\_element}(\text{next}) \parallel \text{is\_cluster\_start}(\text{next}) \parallel$ 
     $sp == \text{orig}$ ) then
11     $\text{set\_elem}(\text{QF}, s, 0);$ 
12    return;
13  else
14    if ( $\text{is\_run\_start}(\text{next})$ ) then
15      do
16         $\text{incr}(\text{QF}, F_Q) \rightarrow F_Q;$ 
17        while ( $\text{is\_occupied}(\text{get\_elem}(\text{QF}, F_Q))$ );
18        if ( $\text{curr\_occupied} \&\& F_Q == s$ ) then
19           $\text{clr\_shifted}(\text{next}) \rightarrow \text{updated\_next};$ 
20         $\text{set\_elem}(\text{QF}, s, \text{curr\_occupied?set\_occupied}$ 
     $(\text{updated\_next}) : \text{clr\_occupied}(\text{updated\_next}));$ 
21         $sp \rightarrow s;$ 
22         $\text{incr}(\text{QF}, sp) \rightarrow sp;$ 
23         $\text{next} \rightarrow \text{curr};$ 
24 if ( $\text{is\_occupied}(T_{F_Q}) \parallel (!\text{QF} \rightarrow \text{QF}_{\text{entries}})$ ) then
25   return true;
26  $\text{start} = \text{find\_run\_index}(\text{QF}, F_Q);$ 
27  $s = \text{start};$ 
28 do
29    $\text{rem} = \text{get\_remainder}(\text{get\_elem}(\text{QF}, s));$ 
30   if  $\text{rem} == F_R$  then
31     break;
32   else if  $\text{rem} > F_R$  then
33     return true;
34    $s = \text{incr}(\text{QF}, s);$ 
35 while ( $\text{is\_continuation}(\text{get\_elem}(\text{QF}, s))$ );
36 if ( $\text{rem} \neq F_R$ ) then
37   return true;
38  $\text{delete\_entry}(\text{QF}, s, F_Q);$ 
39  $\text{QF\_entries} --;$ 
40 return true

```

where k is the number of hash functions, n is the number of elements, m is the length of the filter, e is natural constant.

Since false positives are inevitable in filter structures, we must implement a verification method to validate results. For ensuring the accuracy of results, we maintain a dedicated verify token for each file, linked to its unique ind. Bitmaps, known for their simplicity and efficiency, employ a binary bit array to represent information. The most prevalent bitmap

indexing technique involves associating each bit with a specific element's position: a 1 signifies the element's presence within a set, while a 0 indicates its absence. Then we compress verify token with Roaring Bitmaps [49] and encrypt it. Upon executing a search request, the verify tokens are presented to the DU alongside the other results. Having obtained these tokens, the DU decrypts and authenticates the results using decrypted verify token. The verification process not only reduces FPR but also decreases the quotient filter size, resulting in surprisingly significant space cost savings.

2) *Details of Trinity-II Construction*: In this section, we will briefly introduce **Setup** and **Search**, focusing on **Addition** and **Verification**. **Deletion** phase in Trinity-I and Trinity-II is the same, so it will not be repeated here

Algorithm 5 Setup (1^λ)

Input: $(\{0, 1\}^\lambda, c, \text{DB})$
Output: $(K_{\Sigma}, \sigma; OT_c, \text{EDB})$

```

1  $\text{Setup}(1^\lambda) \rightarrow msk;$ 
2 Randomly generates  $H = \{H_i(\cdot)\}_{i=1}^t;$ 
3  $K_{\Sigma} = \{msk, H\}, \mathbb{G}(msk, c) \rightarrow OT_c, H(K, OT_c) \rightarrow e^c;$ 
4 for space-time node  $O_i = \{p_i, \text{ind}_i\}$  do
5    $H(P(\mathcal{H}(p_i))) \rightarrow e_i, e_i \oplus e^c \rightarrow e_i^c, e_i^c \rightarrow (F_Q^c, F_R^c);$ 
6    $\text{QF}\{H, F_Q^c, F_R^c\} \rightarrow \text{QF}_i;$ 
7    $\text{SHVE.Enc}(msk, \text{"True"}, \text{QF}_i) \rightarrow \text{C}_{\text{SHVE}}(\text{QF}_i);$ 
8    $\{\text{C}_{\text{SHVE}}(\text{QF}_i), \text{ind}_i\}_{i=1}^N \rightarrow \text{EDB};$ 
9 return  $(K_{\Sigma}, \sigma; \text{EDB}, OT_c);$ 

```

Setup($\{0, 1\}^\lambda, 0 \rightarrow c \rightarrow (K_{\Sigma}, \sigma; \text{EDB}, OT_c)$): Given a security parameter λ , a update counter c and a database DB, it returns a primary key msk , t hash functions $H = \{H_i(\cdot)\}_{i=1}^t$, an order token OT_c and an encrypted database EDB. One of the key difference between Trinity-I and Trinity-II is the introduction of counter c , which tracks the number of updates. And the DO generates OT_i by $\mathbb{G}$ with the input of msk and c . Within the salting process, OT_c is itself "salted" using a hash function H . Subsequently, the DU incorporates this salt into the original fingerprints, e_i , resulting in the generation of salted fingerprints, e_i^c . And rest are the same to the Trinity-I. A formal representation of the **Setup** phase is displayed in Algorithm 5.

Search ($K_{\Sigma}, Q, \sigma; \text{EDB}$) $\rightarrow R$: Trinity-I and Trinity-II are highly similar in the search phase, with the only difference being that Trinity-II employs K and e_q^c for salting desalting. Search token e_q^c are like e_i^c but input with a range query instead of space-time node. After the search, all added nodes in QF.Cache are desalted and transferred to QF_i . A formal representation of the **Search** phase is displayed in Algorithm 6.

Update ($K_{\Sigma}, O, add, \sigma; \text{EDB}$) $\rightarrow (K_{\Sigma}, \sigma'; \text{EDB}')$: Since delete operation in Trinity-I and Trinity-II are exactly the same, so here we discuss add operation only. Every time the DO wants to add a space-time node $p = (x, y, z)$ to EDB, first, the DO increases the value of the counter to the match node O . Then the DO starts the "salted" process like **Setup** phase. Next, the DO sends the salted token into the QF.Cache of the server like in Fig. 3. If the number of entries in QF exceeds 20% of its maximum size, expand the QF twice as

Algorithm 6 Search

Input: (K_Σ, Q, σ ; EDB)
Output: R

1 **DU:**
2 $\mathbb{G}(msk, c) \rightarrow ST$;
3 Send ($ST, K, S_{SHVE}(QF_Q)$) to the CS;
4 **CS:**
5 $\mathbb{G}(ST, c) \rightarrow OT_c, H(K, OT_c) \rightarrow e^c$,
 $H(\mathcal{P}(\mathcal{H}(R))) \rightarrow e_q$;
6 $e_q \oplus e^c \rightarrow e_q^c, e_q^c \rightarrow (F_Q^c, F_R^c), QF\{H, F_Q^c, F_R^c\} \rightarrow QF_Q$;
7 $S_{HVE}.KeyGen(msk, QF_Q) \rightarrow S_{SHVE}(QF_Q)$;
8 **if** is_occupied(T_{F_Q}) == false **then**
9 **return** false /* Check if the element at the quotient
 index is occupied. Return false if unoccupied. */
10 $s = \text{find_run_index}(QF, F_Q)$; /* Finds the starting index
 of the run associated with the quotient. */
11 **do**
12 $rem = \text{get_remainder}(\text{get_elem}(QF, s))$;
13 **if** $rem == F_R$ **then**
14 **return** true;
15 **else**
16 $rem > F_R$;
17 **return** false;
18 $s = \text{incr}(QF, s)$ /* Increments the index to move to
 the next element in the run. */
19 **while** (is_continuation($\text{get_elem}(QF, s)$));
20 **return** false;

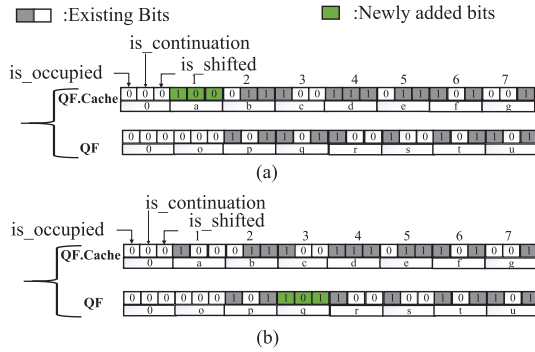


Fig. 3. An example of addition.

the original size. The salted values (e_a^c, F_Q^c, F_R^c) are desalted and sent to QF_i after the next search.

The remaining operations are identical to those in Trinity-I. A formal representation of the **Addition** phase is displayed in Algorithm 7.

Verification: As mentioned earlier in technique overview, in order to maintain a low false positive rate of about 0.01%, it is customary to set the ratio N/m at 20, and the parameter t at 14. This implies that for a database with a capacity of 5 million space-time nodes, a filter size of 100 million would be required. Such a configuration results in a considerable waste of storage space. But with the help of verification, we do not require such low FPR, we only have to keep a basic accuracy of quotient filter that doesn't affect normal search function. Based on verification, we set $FPR \leq 6\%$, and $t = -\ln(p)/\ln(2)$,

Algorithm 7 Addition

Input: (K_Σ, O, add, σ ; EDB)
Output: (K_Σ, σ' ; EDB')

1 **DO:**
2 $\mathbb{G}(msk, c) \rightarrow OT_c$;
3 $H(K, OT_c) \rightarrow e^c, H(\mathcal{P}(\mathcal{H}(p_a))) \rightarrow e_a$;
4 $e_a \oplus e^c \rightarrow e_a^c, e_a^c \rightarrow (F_Q^c, F_R^c)$;
5 Send (c, F_Q^c, F_R^c) to the CS;
6 **CS:**
7 (c, F_Q^c, F_R^c) $\rightarrow$ QF.Cache;
8 $prev = \text{get_elem}(QF_i, s)$;
9 $empty = \text{is_empty_element}(prev)$;
10 **do**
11 **if** (!empty) **then**
12 $prev = \text{set_shifted}(prev)$;
13 **if** (is_occupied_element($prev$)) **then**
14 $curr = \text{set_occupied_element}(curr)$;
15 $prev = \text{clr_occupied_element}(prev)$;
16 $\text{set_element}(QF, s, curr)$;
17 $curr = prev, s = \text{incr}(QF, s), prev = \text{get_elem}(QF, s)$
 ;
18 $empty = \text{is_empty_element}(prev)$;
19 **while** (!empty);
20 **if** ($QF_i_entries \geq \frac{1}{5} QF_i_max_size$) **then**
21 $i++$; /* Expand the original QF_i to QF_{i+1} when
 QF_i exceeds half capacity. */
22 $T_{F_Q} = \text{get_elem}(QF_i, F_Q^c)$;
23 $entry = (F_R^c \ll 3) \& \sim 7$;
24 **if** (is_empty_element(T_{F_Q})) **then**
25 $\text{set_elem}(QF_i, F_Q^c, \text{set_occupied}(entry))$;
26 $QF_entries++$;
27 **return** true
28 **if** (is_occupied(T_{F_Q})) **then**
29 $\text{set_elem}(QF_i, F_Q^c, \text{set_occupied}(T_{F_Q}))$;
30 $start = \text{find_run_index}(QF_i, F_Q^c)$;
31 $s = \text{start}$;
32 **if** $s \neq F_Q^c$ **then**
33 $entry = \text{set_shifted}(entry)$;
34 $\text{insert_into}(QF_i, s, entry)$;
35 $QF_entries++$;
36 **return** true;

we have 4.05. And assume we have 1 millions elements in the filter, we should set the length of filter as $N = mt/\ln(2)$, the number is 5.85 millions. So we set $N/m = 6$, $t = 4$, and the FPR is 5.56%. Consequently, both storage and computational costs decrease precipitously. Next, I will formally describe the Trinity-II. Verification Process, which consists of two distinct phases: Setup and Search.

- **Setup:** An additional verify array is generated for each space-time node $V(\mathcal{H}(p_i)) \rightarrow va_i$. An extra secret key sk is generated to encrypt verify arrays. Then we compress the verify array $\text{comp}(vt_i) \rightarrow vt'_i$ and encrypt it $\text{Enc}(va'_i) \rightarrow$

vt_i . Finally, we store vt_i associated with the ind_i . The rest procedure are the same to the Trinity-II.

- **Search:** The CS sends verify token vt_i along with other results to the DU. Once receiving results, the DU decrypts the vt_i and verifies the token according to the query, if it matches, keep the results; if not, remove the index.

VI. SECURITY ANALYSIS

This section delves into security analysis of our scheme. We begin by defining a leakage function, and then proposing a rigorous proof of the construction's security.

The leakage function, denoted as $\mathcal{L}$, encompasses leakage from three distinct phases: setup, search, and update operations. $\mathcal{L} = \{\mathcal{L}^{Stp}, \mathcal{L}^{Srch}, \mathcal{L}^{Updt}\}$. Meanwhile, the leakage function can also be explained as follows:

Access pattern: The access pattern, denoted as ap , signifies the specific locations of documents that align with the query.

Size pattern: The size pattern, denoted as $\mathbb{S}$, reflects in the number of objects m .

Theorem 1: Define the leakage function $\mathcal{L}_1$ of Trinity-I, if it can be described as following:

$$\mathcal{L}_1 = \{\mathbb{S}(DB), ap\},$$

Trinity-I is Indistinguishability under Selective Chosen-Plaintext Attack (IND-SCPA) secure if SHVE is IND-SCPA secure.

Proof: The proof of Theorem 1 is established through a simulation-based approach. Assuming that the adversary $\mathcal{A}$ cannot differentiate between the output of the real game and the ideal game, we can conclude that no leakage exists beyond $\mathcal{L}_1$. All spatio-temporal data are encrypted by SHVE which is proved in [13]. The security of Trinity-I is based on the security of SHVE. So the proof of Theorem 1 leverages a simulation as follows:

- **Setup:** The adversary $\mathcal{A}$ transmits a chosen database, denoted as $DB = \{\mathbf{p}_i, \mathbf{ind}_i\}_{i=1}^N$, to the challenger $\mathcal{C}$. The challenger $\mathcal{C}$ generates both a set of t random hash functions H and a secret key msk , maintaining the secrecy of $K_{\Sigma} = \{msk, H\}$.
- **Phase 1:** The adversary $\mathcal{A}$ proceeds to adaptively select a series of queries, each represented as Q_j , where $j \in [q_1]$. In response to each query, $\mathcal{C}$ encodes the range query Q_j into $P(\mathcal{H}(Q_j))$. Subsequently, for each prefix element pe_k within $P(\mathcal{H}(Q_j))$ (where $1 \leq k \leq \beta$), $\mathcal{C}$ constructs a quotient filter comprises $(H(p_i), F_Q, F_R)$, and then encrypts it with SHVE, with '0' positions being substituted with '★' and '1' positions remaining unaltered. Finally, $\mathcal{C}$ transmits the search token ST_j to $\mathcal{A}$.
- **Challenge:** $\mathcal{C}$ randomly selects a bit, denoted as b , from the set $\{0, 1\}$. For each space-time node O_i , $\mathcal{C}$ constructs a corresponding quotient filter, $QF\{H, (e_i, F_Q, F_R)\} \rightarrow QF_i$. Subsequently, $\mathcal{C}$ offers $\mathcal{A}$ with the challenge encrypted database EDB , constructed by $SHVE.Enc(msk, QF_i) \rightarrow C_{SHVE}(QF_i)$ for $i \in [N]$.
- **Phase 2:** $\mathcal{A}$ repeats the **Phase 1** procedure and receives ST_j for $q_1 + 1 \leq j \leq q_2$.
- **Guess:** $\mathcal{A}$ takes a guess of b .

Due to the utilization of SHVE for encryption within our Trinity-I, the indistinguishability of indexes and trapdoors within the Trinity-I directly inherits the indistinguishability property of SHVE. Notably, the security game associated with our Trinity-I is essentially simulated by q' instances of SHVE, where q' denotes the total number of SHVE instances engaged in the game. As a result, the ability of adversary $\mathcal{A}'$ to distinguish between two SHVE encrypted values would directly enable it to distinguish between indexes and trapdoors within the Trinity-I. This relationship can be succinctly expressed as follows:

$$\text{Adv}_{\text{Trinity-I}, \mathcal{A}}^{\text{IND-SCPA}}(1^\lambda) \leq \text{Adv}_{\text{SHVE}, \mathcal{A}'}^{\text{IND-SCPA}}(1^\lambda) \leq q' \cdot \text{negl}(\lambda).$$

Since Trinity-II also utilizes SHVE as its encryption tool, its security can be proven using the same approach. $\square$

We formally define forward security for Trinity-II, where CPRF and "salts" work in tandem to achieve this property. This combination effectively thwarts adaptive file injection attacks, as rigorously proven by the following demonstration.

Forward Security. Forward security prevents information leakage during the addition of new data in dynamic updates. For clarity, we adopt the precise definition from [36].

Theorem 2: If the function $\mathcal{L}^{FS} = \{\mathcal{L}^{Updt}\}$ is defined as follows, Trinity-II can be considered as forward-secure:

$$\mathcal{L}^{Updt}(op, in) = \mathcal{L}'(op, (ind_i, c)). \quad (1)$$

The function $\mathcal{L}'$ is stateless and c represents the number of updated keywords for the updated file ind_i .

Proof: First, spatio-temporal data is encrypted locally by the DO using SHVE before being transmitted to the CS. Then, Trinity-II uses SHVE to ensure document security while keeping the key private. In this way, the primary key msk of SHVE is held only by the DO, thus preventing unauthorized access by the CS. Indices undergo encryption locally before being uploaded to the CS for security and confidentiality. The quotient filter stores only fingerprints, quotients and remainders. And ST generation encrypts each range query using SHVE, CPRF and "salts", ensuring confidentiality. the DO incorporates "salts" alongside SHVE when adding nodes to the quotient filter, effectively preventing the CS from inferring keyword information from past search tokens. $\square$

VII. PERFORMANCE EVALUATION

We implemented Trinity in C++,¹ with SHVE utilizing multi-threading technology, CPRF implemented via HMAC256, and Murmur serving as the hash function. In this section, a comprehensive comparison is made between the construction performance, volume size, token generation efficiency, search capability, addition performance, and deletion performance of Trinity-I and Trinity-II, as compared to those of SKSE-II [23], GRS-II [12], and DSSE_{SKQ} [14]. We conducted experiments on an Ubuntu 20.04.5 LTS system using an Intel Xeon Gold 6226R processor. This processor has a base clock speed of 2.90 GHz. The system featured 64*8 = 512 GB of RAM. Network bandwidth used during these experiments was set at 100 Mbps.

¹<https://github.com/eulermachine/trinity/>

TABLE II
NOTATION DESCRIPTION FOR TRINITY

Notations	Descriptions
QF	Quotient Filter
ξ, ξ'	random values between (0, 1)
$\vec{p}$	spatio-temporal object vector
Q	search query
N	size of Quotient Filter
OT_c	order token corresponding to the current count
e_{c+1}	salted token
M_{1_c}, M_{2_c}	salted matrices
p_i	encoded spatio-temporal data
O	space-time node
ind_i	the i -th file identifier.
$H()$	hash function
$\mathcal{H}()$	Hilbert curve encode
$P()$	prefix of the nodes
$\mathcal{P}()$	prefix of the range
e_i	fingerprint of file identifier
F_Q	quotient of fingerprint
F_R	remainder of fingerprint
ST_{non}	Search token for non-leaf nodes
ST_L	Search token for leaf nodes

A. Theoretical Analysis

Table III provides a comprehensive comparison of the time and space complexity of GRS-II, DSSE_{SKQ}, SKSE-II, Trinity-I and Trinity-II. Since we use quotient filter as search structure and there are k queries sent by the DUs, the Trinity-I's computation complexity of search is $km \cdot \log m$. So is computation complexity of update. As for communication complexity of search, it is $\mathcal{O}(k\lambda m \cdot \log m)$. The communication complexity of update is $\mathcal{O}(k\lambda m)$. Trinity-II differs in two aspects: the use of “salts” and QF.Cache. The employed GGM-tree requires $\mathcal{O}(\log c)$ computation time to retrieve ST or OT. So we add $\mathcal{O}(\log c)$ and $\mathcal{O}(\lambda \log c)$ to the computation complexity and communication complexity of search in Trinity-II. And since we set up a QF.Cache, there is an extra size of EDB, but we set the size of QF.Cache to $\mathcal{O}(\log m)$, which is negligible. As mentioned earlier, the storage overhead has been reduced from $20\times$ redundancy to $4.05\times$ redundancy, achieving an 80% reduction in storage cost. However, this optimization increases the probability of hash collisions from 0.01% to 5.56%. Nevertheless, by employing cryptographic verification techniques, we effectively mitigate any erroneous outcomes caused by hash collisions. In essence, we have transformed 80% of the storage overhead into negligible computational and bandwidth costs associated with verification.

B. Datasets

We use a spatio-temporal dataset from Yelp to evaluate the performance. We selected 1,007,016 timestamped locations, with some locations appearing multiple times at different timestamps. In the context of fine-grained spatiotemporal data, we refer to the logistics transportation scenario and set the spatial granularity to 10 meters and the temporal granularity to 5 minutes [50].

C. Setup Performance

Setup latency: The setup latency of Trinity-I and Trinity-II remains stable and increases linearly, as do the setup latency

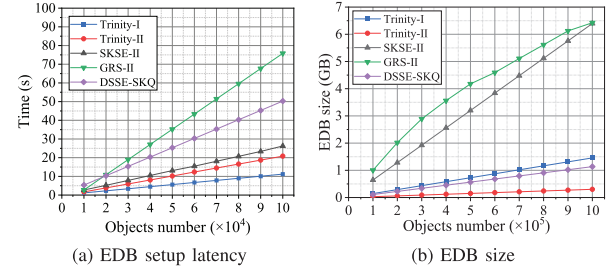


Fig. 4. Trinity-I vs Trinity-II, SKSE-II, GRS-II and DSSE_{SKQ} setup performance.

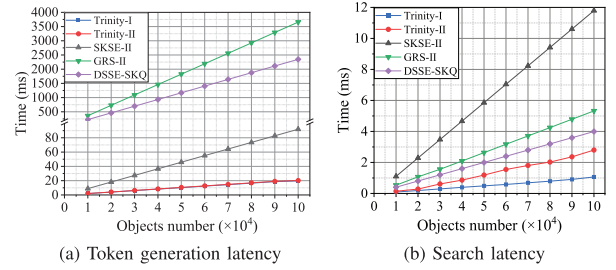


Fig. 5. Trinity-I vs Trinity-II, SKSE-II, GRS-II and DSSE_{SKQ} search performance.

of GRS-II, DSSE_{SKQ}, and SKSE-II. For instance, as shown in Figure 4(a), Trinity-I takes about 11 seconds to build an EDB for 100,000 data points, which is significantly less than the time taken by Trinity-II, which takes 20 seconds. Meanwhile, SKSE-II, GRS-II, and DSSE_{SKQ} currently take 26 seconds, 75 seconds, and 50 seconds respectively to construct an EDB of 100,000 data points. To ensure accuracy, each method was tested 100 times, with the average value serving as the final result.

EDB Size: The size of Trinity-I and Trinity-II expands proportionally to the number of data points, as does the size of SKSE-II, GRS-II and DSSE_{SKQ}. As illustrated in Figure 4(b), when handling 100,000 data points, Trinity-I and DSSE_{SKQ} occupy EDB sizes of 1.456 GB and 1.136 GB respectively, which are relatively similar and notably five times smaller than the EDB sizes consumed by GRS-II (6.395 GB) and SKSE-II (6.424 GB). However, Trinity-II boasts a much more compact EDB size of 0.30146 GB due to its efficient roar bitmap architecture, which dramatically reduces storage cost.

D. Search Performance

Token Generation Performance: In evaluating the token generation cost of Trinity-I and Trinity-II, we compare their token generation times with those of Figure 5(a) for SKSE-II, GRS-II, and DSSE_{SKQ}. The evaluation tested different numbers of entries, ranging from 10,000 to 100,000 space-time nodes. Trinity-I and Trinity-II exhibit minimal differences in token generation cost, with respective times of 19.748 ms and 20.198 ms per token generation. In terms of encryption methods, SKSE-II employs HVE, while GRS-II and DSSE_{SKQ} utilize ASHE. Their corresponding token generation times are 92.024 ms, 3658.275 ms, and 2347.0956 ms, respectively. To ensure accuracy, each method was tested 100 times, with the average value serving as the final result.

TABLE III
COMPLEXITY COMPARISONS

Scheme	Computation		Communication		Storage size
	Search	Update	Search	Update	
GRS-II	$\mathcal{O}(\log R \cdot N)$	$\mathcal{O}(2^t N)$	$\mathcal{O}(\log R \cdot N)$	$\mathcal{O}(2^t N)$	$\mathcal{O}(2^t N)$
DSSE _{SKQ}	$\mathcal{O}(a_w(R + k))$	$\mathcal{O}(\log N + \log m)$	$\mathcal{O}((1 + k)l)$	$\mathcal{O}((\log N + \log m)(2\lambda + l))$	$\mathcal{O}(a_w(\log N + \log m)(2\lambda + l))$
SKSE-II	$\mathcal{O}(\bar{x}km \cdot \log m)$	N/A	$\mathcal{O}(\bar{x}k\lambda m \cdot \log m)$	N/A	$\mathcal{O}(m\lambda N)$
Trinity-I	$\mathcal{O}(km \cdot \log m)$	$\mathcal{O}(k)$	$\mathcal{O}(k\lambda m \cdot \log m)$	$\mathcal{O}(k\lambda)$	$\mathcal{O}(m\lambda N)$
Trinity-II	$\mathcal{O}(k(m \cdot \log m + \log c))$	$\mathcal{O}(k)$	$\mathcal{O}(k\lambda(m \cdot \log m + \log c))$	$\mathcal{O}(k\lambda)$	$\mathcal{O}(m\lambda N)$

Notes. N is the number of data points in the database (i.e. the number of entries), M is the number of keywords in the database, m is the size of database, $|U|$ is the size of each ciphertext, λ is a security parameter, k is the number of returned results range, $|k|$ is the size of key K , $|\varepsilon|$ is the size of random vector ε , $\bar{x}$ is the average number of the nodes at each level that traversed by the CS, c is the node number of GGM tree and R is search range radius. 2^t is the size of the binary tree.

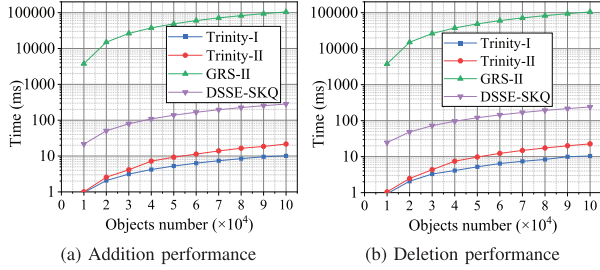


Fig. 6. Trinity-I vs Trinity-II, GRS-II and DSSE_{SKQ} update performance.

Search Latency: In evaluating the efficiency of Trinity-I and Trinity-II, we compare their search times to those of SKSE-II, GRS-II, and DSSE_{SKQ}, as depicted in Figure 5(b). The number of entries, which refers to space-time objects, ranges from 10,000 to 100,000 in increments of 10,000. The search range is set to 100 m^2 in 5 minutes. As the number of entries increases, the search time cost per entry in Trinity-II increases once the number exceeds 10% of its current capacity. This results in a change from a search time cost per entry of 14.262 ns when there are 10,000 entries to 27.994 ns when there are 100,000 entries.

In contrast, the average search time per entry for Trinity-I, SKSE-II, GRS-II, and DSSE_{SKQ} is respectively 10.62 ns, 110.1 ns, 53.162 ns, and 40.1 μ s. Consequently, it can be inferred that Trinity-I outperforms Trinity-II, being 2.63 times faster when there are 100,000 entries. Additionally, Trinity-II is also 4.21 times faster than SKSE-II, 1.9 times faster than GRS-II, and 1.426 times faster than DSSE_{SKQ}.

E. Update Performance

Addition Performance: The addition time for Trinity-I, Trinity-II and DSSE_{SKQ} increases linearly, while for GRS-II the addition time increases as a logarithmic function. However, as shown in Figure 6(a), Trinity-I, Trinity-II and DSSE_{SKQ} at 100,000 points have an addition time of 10.09 ms, 21.59 ms and 282.77 ms respectively, which is negligible compared with the 105,301 ms of GRS-II. This is because each addition in GRS-II is also an update of the entire binary tree. Also, SKSE-II will not be discussed here because it does not support dynamic updates.

Deletion Performance: Similar to the add operation, the deletion time for Trinity-I, Trinity-II and DSSE_{SKQ} increases linearly, while for GRS-II, the deletion time increases as logarithmically. As shown in Figure 6(b), Trinity-I, Trinity-II and DSSE_{SKQ} have deletion times of 10.44 ms, 22.91 ms and 243.53 ms at 100,000 points, respectively, which are negligible

compared to the 105,277 ms of GRS-II. This is because each deletion in GRS-II is also an update of the entire binary tree. Again, SKSE-II will not be discussed here because it does not support dynamic updates.

VIII. CONCLUSION

We propose a novel spatio-temporal data DSSE scheme Trinity-I that supports efficient dynamic updates, and automatic scalability. Trinity-I utilizes Hilbert curves and quotient filters to achieve spatio-temporal range query, implementing IND-SCPA security based on SHVE technology. Additionally, Trinity-I can perform millions of data retrievals within just a few milliseconds. Our solution simultaneously addresses the issues of low query efficiency, scalability challenges, and lack of deletion support, making it more suitable for tasks in spatio-temporal scenarios. But Trinity-I is storage expensive and vulnerable to file injection attacks. So we propose a Trinity-II to solve those questions. Our Trinity-II is forward-secure and storage-saving. Our Trinity-II saves 80% storage cost than Trinity-I, and eliminate false positive by verification. However, the reduced storage cost also results in a smaller capacity for the QF, making hash collisions more likely. Consequently, compared to Trinity-I, Trinity-II exhibits increased query latency and update latency. However, experimental data consistently demonstrate that our proposed solution Trinity-II, outperforms existing solutions in terms of query efficiency, update efficiency, and storage cost.

Trinity-II only provides forward security to protect against attacks from [11] and [32] caused by the add operation. While the leakage caused by the delete operation is not taken into account, and the backward security for the spatio-temporal data security retrieval scheme requires further improvement. Attacks from [11] and [32] are not able to extend to deletion operations, deletion operations only concern about backward security.

Limitation. Without backward security, attackers can analyze the identifiers of deleted files (such as timestamp, geographic location coding, data structure location, etc.): presume that the file once existed; guess its relevance to other documents; further infer the content type of the file or the user's behavior pattern (for example, a user deleted data at a certain time in a certain place, which may mean that this person has visited the place).

Future work. ① Our future work could explore bandwidth optimization techniques, such as incremental token transmission or leveraging lightweight compression algorithms to reduce the overhead of Verify Tokens. ② We also intend to

incorporate backward security in our future design. Currently, two mainstream approaches are commonly adopted to achieve this goal. The first approach introduces the notion of time or update counters, partitioning operations into distinct epochs. Both the client and the server (or in some cases, only the client) maintain an epoch counter that evolves over time. The keys used to encrypt index entries and generate search tokens are updated in accordance with the epoch progression, thereby ensuring that previously exposed keys cannot compromise future data. The second approach focuses on actively modifying or removing index entries. Instead of merely marking documents as deleted, this method requires the system to genuinely update or erase the associated index entries on the server, ensuring that they cannot be matched by any future search tokens. Representative techniques under this paradigm include constructions based on Puncturable Pseudo-Random Functions and Updatable Encryption. These strategies provide complementary trade-offs between performance and security, and will be evaluated for integration in our subsequent scheme.

REFERENCES

- [1] A. Hern. (Jan. 28, 2018). *Fitness Tracking App Strava Gives Away Location of Secret U.S. Army Bases*. Guardian. Accessed: Apr. 8, 2025. [Online]. Available: <https://www.theguardian.com/world/2018/jan/28/fitness-tracking-app-gives-away-location-of-secret-us-army-bases>
- [2] E. Stefanov, C. Papamanthou, and E. Shi, "Practical dynamic searchable encryption with small leakage," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, vol. 71, 2014, pp. 72–75.
- [3] Y. Cao, Y. Xiao, L. Xiong, L. Bai, and M. Yoshikawa, "Protecting spatiotemporal event privacy in continuous location-based services," *IEEE Trans. Knowl. Data Eng.*, vol. 33, no. 8, pp. 3141–3154, Aug. 2021.
- [4] X. Huang, Y. Gao, X. Gao, and G. Chen, "NETR-tree: An efficient framework for social-based time-aware spatial keyword query," in *Proc. IEEE Conf. Web Services (ICWS)*, Sep. 2021, pp. 198–207.
- [5] X. Zhu, E. Ayday, and R. Vitenberg, "A privacy-preserving framework for outsourcing location-based services to the cloud," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 1, pp. 384–399, Jan./Feb. 2021.
- [6] B. Wang, Y. Hou, and M. Li, "QuickN: Practical and secure nearest neighbor search on encrypted large-scale data," *IEEE Trans. Cloud Comput.*, vol. 10, no. 3, pp. 2066–2078, Jul./Sep. 2022.
- [7] R. Guo, B. Qin, Y. Wu, H. Chen, and C. Li, "Search geometric ranges efficiently as keywords over encrypted spatial data," *High-Confidence Comput.*, vol. 2, no. 2, Jun. 2022, Art. no. 100058.
- [8] Y. Yang, Y. Miao, K. R. Choo, and R. H. Deng, "Lightweight privacy-preserving spatial keyword query over encrypted cloud data," in *Proc. IEEE 42nd Int. Conf. Distrib. Comput. Syst. (ICDCS)*, Jul. 2022, pp. 392–402.
- [9] Y. Miao et al., "Efficient privacy-preserving spatial range query over outsourced encrypted data," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 3921–3933, 2023.
- [10] X. Zhang, D. Mu, and J. Zhao, "Attribute-based keyword search encryption for power data protection," *High-Confidence Comput.*, vol. 3, no. 2, 2023, Art. no. 100115.
- [11] Y. Zhang, J. Katz, and C. Papamanthou, "All your queries are belong to us: The power of file-injection attacks on searchable encryption," in *Proc. USENIX Secur. Symp.*, 2016, pp. 707–720.
- [12] S. K. Kermanshahi et al., "Geometric range search on encrypted data with forward/backward security," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 1, pp. 698–716, Jan. 2022.
- [13] J. Li, J. Ma, Y. Miao, F. Yang, X. Liu, and K.-K. R. Choo, "Secure semantic-aware search over dynamic spatial data in VANETs," *IEEE Trans. Veh. Technol.*, vol. 70, no. 9, pp. 8912–8925, Sep. 2021.
- [14] X. Wang, J. Ma, X. Liu, Y. Miao, Y. Liu, and R. H. Deng, "Forward/backward and content private DSSE for spatial keyword queries," *IEEE Trans. Dependable Secure Comput.*, pp. 1–13, 2022.
- [15] Z. Li, J. Ma, Y. Miao, X. Wang, J. Li, and C. Xu, "Enabling efficient privacy-preserving spatiotemporal location-based services for smart cities," *IEEE Internet Things J.*, vol. 11, no. 3, pp. 5288–5300, Feb. 2024.
- [16] M. Etemad, A. K  p   , C. Papamanthou, and D. Evans, "Efficient dynamic searchable encryption with forward privacy," *Proc. Privacy Enhanc. Technol.*, vol. 1, pp. 5–20, Apr. 2022.
- [17] B. Wang, M. Li, and L. Xiong, "FastGeo: Efficient geometric range queries on encrypted spatial data," *IEEE Trans. Dependable Secure Comput.*, vol. 16, no. 2, pp. 245–258, Mar. 2019.
- [18] H. Dou et al., "Dynamic searchable symmetric encryption with strong security and robustness," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 2370–2384, 2024.
- [19] X. Li, Y. Zhu, J. Wang, and J. Zhang, "Efficient and secure multi-dimensional geometric range query over encrypted data in cloud," *J. Parallel Distrib. Comput.*, vol. 131, pp. 44–54, Sep. 2019.
- [20] Z. Zheng, Z. Cao, and J. Shen, "Practical and secure circular range search on private spatial data," in *Proc. IEEE 19th Int. Conf. Trust, Secur. Privacy Comput. Commun. (TrustCom)*, Dec. 2020, pp. 639–645.
- [21] B. Wang, Y. Hou, M. Li, H. Wang, and H. Li, "Maple: Scalable multi-dimensional range search over encrypted cloud data with tree-based index," in *Proc. 9th ACM Symp. Inf. Comput. Commun. Secur.*, Jun. 2014, pp. 111–122.
- [22] N. Cui, J. Li, X. Yang, B. Wang, M. Reynolds, and Y. Xiang, "When geo-text meets security: Privacy-preserving Boolean spatial keyword queries," in *Proc. IEEE 35th Int. Conf. Data Eng. (ICDE)*, Apr. 2019, pp. 1046–1057.
- [23] X. Wang, J. Ma, F. Li, X. Liu, Y. Miao, and R. H. Deng, "Enabling efficient spatial keyword queries on encrypted data with strong security guarantees," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 4909–4923, 2021.
- [24] S. Zhang, S. Ray, R. Lu, Y. Guan, Y. Zheng, and J. Shao, "Efficient and privacy-preserving spatial keyword similarity query over encrypted data," *IEEE Trans. Dependable Secure Comput.*, pp. 1–16, 2022.
- [25] P. S. Almeida, C. Baquero, N. Pregui  a, and D. Hutchison, "Scalable Bloom filters," *Inf. Process. Lett.*, vol. 101, no. 6, pp. 255–261, 2007.
- [26] P. Pandey, A. Conway, J. Durie, M. A. Bender, M. Farach-Colton, and R. Johnson, "Vector quotient filters: Overcoming the time/space trade-off in filter design," in *Proc. Int. Conf. Manage. Data*, Jun. 2021, pp. 1386–1399.
- [27] J. K. Mullin, "A second look at Bloom filters," *Commun. ACM*, vol. 26, no. 8, pp. 570–571, 1983.
- [28] R. Li and A. X. Liu, "Adaptively secure and fast processing of conjunctive queries over encrypted data," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 4, pp. 1588–1602, Apr. 2022.
- [29] X. Li et al., "VRFMS: Verifiable ranked fuzzy multi-keyword search over encrypted data," *IEEE Trans. Services Comput.*, vol. 16, no. 1, pp. 698–710, Jan. 2023.
- [30] Q. Tong, Y. Miao, J. Weng, X. Liu, K. R. Choo, and R. H. Deng, "Verifiable fuzzy multi-keyword search over encrypted data with adaptive security," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 5, pp. 5386–5399, May 2023.
- [31] L. Fan, P. Cao, J. Almeida, and A. Z. Broder, "Summary cache: A scalable wide-area Web cache sharing protocol," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 28, no. 4, pp. 254–265, 1998.
- [32] D. Cash, P. Grubbs, J. Perry, and T. Ristenpart, "Leakage-abuse attacks against searchable encryption," in *Proc. 22nd ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2015, pp. 668–679.
- [33] K. S. Kim, M. Kim, D. Lee, J. H. Park, and W.-H. Kim, "Forward secure dynamic searchable symmetric encryption with efficient updates," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2017, pp. 1449–1463.
- [34] M. Xu et al., "FileDES: A secure, scalable and succinct decentralized encrypted storage network," in *Proc. IEEE Conf. Comput. Commun. (IEEE INFOCOM)*, May 2024, pp. 261–270.
- [35] C. Zuo, S.-F. Sun, J. K. Liu, J. Shao, J. Pieprzyk, and L. Xu, "Forward and backward private DSSE for range queries," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 1, pp. 328–338, Jan. 2022.
- [36] R. Bost, " $\sum O\phi O\phi$: Forward secure searchable encryption," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2016, pp. 1143–1154.
- [37] X. Song, C. Dong, D. Yuan, Q. Xu, and M. Zhao, "Forward private searchable symmetric encryption with optimized I/O efficiency," *IEEE Trans. Dependable Secure Comput.*, vol. 17, no. 5, pp. 912–927, Sep. 2020.

- [38] F. Li et al., "Towards efficient verifiable Boolean search over encrypted cloud data," *IEEE Trans. Cloud Comput.*, vol. 11, no. 1, pp. 839–853, Jan. 2023.
- [39] C. Guo, W. Li, X. Tang, K. R. Choo, and Y. Liu, "Forward private verifiable dynamic searchable symmetric encryption with efficient conjunctive query," *IEEE Trans. Dependable Secure Comput.*, pp. 1–17, 2023.
- [40] B. Wang, M. Li, H. Wang, and H. Li, "Circular range search on encrypted spatial data," in *Proc. IEEE Conf. Commun. Netw. Secur. (CNS)*, Sep. 2015, pp. 182–190.
- [41] F. Kerschbaum and A. Schroepfer, "Optimal average-complexity ideal-security order-preserving encryption," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2014, pp. 275–286.
- [42] P. Grubbs, K. Sekniqi, V. Bindshaedler, M. Naveed, and T. Ristenpart, "Leakage-abuse attacks against order-revealing encryption," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2017, pp. 655–672.
- [43] R. Li, A. X. Liu, Y. Liu, H. Xu, and H. Yuan, "Insecurity and hardness of nearest neighbor queries over encrypted data," in *Proc. IEEE 35th Int. Conf. Data Eng. (ICDE)*, 2019, pp. 1614–1617.
- [44] W. K. Wong, D. W.-L. Cheung, B. Kao, and N. Mamoulis, "Secure kNN computation on encrypted databases," in *Proc. ACM Int. Conf. Manag. Data*, 2009, pp. 139–152.
- [45] D. Hilbert and D. Hilbert, "Über die stetige abbildung einer linie auf ein flächenstück," in *Dritter Band: Analysis Grundlagen der Mathematik Physik Verschiedenes: Nebst Einer Lebensgeschichte*. Berlin, Germany: Springer, 1935, pp. 1–2.
- [46] R. Curtmola, J. Garay, S. Kamara, and R. Ostrovsky, "Searchable symmetric encryption: Improved definitions and efficient constructions," in *Proc. 13th ACM Conf. Comput. Commun. Secur.*, 2006, pp. 79–88.
- [47] A. X. Liu and F. Chen, "Privacy preserving collaborative enforcement of firewall policies in virtual private networks," *IEEE Trans. Parallel Distrib. Syst.*, vol. 22, no. 5, pp. 887–895, May 2011.
- [48] Y. Liang, J. Ma, Y. Miao, D. Kuang, X. Meng, and R. H. Deng, "Privacy-preserving Bloom filter-based keyword search over large encrypted cloud data," *IEEE Trans. Comput.*, vol. 72, no. 11, pp. 3086–3098, Nov. 2023.
- [49] S. Chambl, D. Lemire, O. Kaser, and R. Godin, "Better bitmap performance with roaring bitmaps," *Softw., Pract. Exp.*, vol. 46, no. 5, pp. 709–719, May 2016.
- [50] Y. Liang et al., "UrbanFM: Inferring fine-grained urban flows," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Disc. Data Min.*, 2019, pp. 3132–3142.



Minghui Xu (Member, IEEE) received the B.S. degree in physics from Beijing Normal University, Beijing, China, in 2018, and the Ph.D. degree in computer science from George Washington University, Washington, DC, USA, in 2021. He is currently a Professor with the School of Computer Science and Technology, Shandong University, China. His research interests include blockchain, distributed computing, and applied cryptography.



Xiangyu Wang (Member, IEEE) received the B.E. and Ph.D. degrees from Xidian University, Xi'an, China, in 2017 and 2021, respectively. He is currently an Associate Professor with the School of Cyber Engineering, Xidian University. His research interests include big data security, cloud security, and applied cryptography. He received the Outstanding Doctoral Dissertation Award from China Institute of Communications in 2022.



Yinbin Miao (Senior Member, IEEE) received the Ph.D. degree from the Department of Telecommunication Engineering, Xidian University, Xi'an, China, in 2016. He was a Post-Doctoral Fellow with Nanyang Technological University, Singapore, from 2018 to 2019. He is currently a Lecturer with the Department of Cyber Engineering, Xidian University. His current research interests include information security and applied cryptography.



Zhijun Li received the B.E. degree from the School of Software Engineering, Dalian University of Technology, Dalian, China, in 2018, and the Ph.D. degree in cyberspace security from Xidian University, Xi'an, China, in 2024. He is currently a Post-Doctoral Researcher with the School of Computer Science and Technology, Shandong University, China. His current research interests include cloud computing security, edge computing security, and applied cryptography.



Jianfeng Ma (Member, IEEE) received the Ph.D. degree in computer software and telecommunication engineering from Xidian University, Xi'an, China, in 1995. He was a Research Fellow with Nanyang Technological University, Singapore, from 1999 to 2001. He is currently a Professor and a Ph.D. Supervisor with the Department of Cyber Engineering, Xidian University. His current research interests include information and network security, wireless and mobile computing systems, and computer networks.



Kuizhi Liu received the B.S. degree from Nankai University, Tianjin, China, in 2021. He is currently pursuing the Ph.D. degree with the School of Cyber Engineering, Xidian University. His current research interests include cloud computing security, database security, and data applied cryptography.



Xiuzhen Cheng (Fellow, IEEE) received the M.S. and Ph.D. degrees in computer science from the University of Minnesota-Twin Cities in 2000 and 2002, respectively. She is currently a Professor with the School of Computer Science and Technology, Shandong University. She worked as the Program Director of U.S. National Science Foundation (NSF) from April 2006 to October 2006 (full-time) and from April 2008 to May 2010 (part-time). Her current research interests include wireless and mobile security, cyber physical systems, wireless and mobile computing, sensor networking, and algorithm design and analysis. She is a member of ACM. She received the NSF CAREER Award in 2004. She also has chaired several international conferences. She has served on the editorial boards for several technical journals and the technical program committees for various professional conferences/workshops.